SESSION 6 LAB PROGRAMS

C.Devasri 192521422

24.Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string

Example 1:

Input: words = ["mass","as","hero","superhero"]

Output: ["as","hero"]

Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".

["hero","as"] is also a valid answer.

Example 2:

Input: words = ["leetcode","et","code"]

Output: ["et","code"]

Explanation: "et", "code" are substring of "leetcode".

Example 3:

Input: words = ["blue","green","bu"]

Output: []

Explanation: No string of words is substring of another string.

CODE:

```
#include <stdio.h>
#include <string.h>

int main() {
   int n, i, j, first = 1;
   char w[50][50];
   printf("Enter number of words: ");
   scanf("%d", &n);
   printf("Enter %d words: ", n);
   for (i = 0; i < n; i++) scanf("%s", w[i]);

   printf("Output: [");
   for (i = 0; i < n; i++) {
      for (j = 0; j < n; j++) {
         if (i != j && strstr(w[j], w[i])) {  /* inside */
            if (!first) printf(",");
            printf("\"%s\"", w[i]);
            first = 0;
            break;
         }
      }
   }
   printf("]\n");
   return 0;
}
```

OUTPUT:

```
Enter number of words: 4
Enter 4 words: mass as hero superhero
Output: ["as","hero"]
```

```
Enter number of words: 3
Enter 3 words: leetcode et code
Output: ["et","code"]
```

```
Enter number of words: 3
Enter 3 words: blue green bu
Output: []
```

Time Complexity: O($n^2 \times L$), where L is the word length | Space Complexity: O(1) (apart from the output)

25. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

Input:

A list or array of points represented by coordinates (x, y).

Points: [(1, 2), (4, 5), (7, 8), (3, 1)]

Output:

The two points with the minimum distance between them.

The minimum distance itself.

Closest pair: (1, 2) - (3, 1) Minimum distance: 1.4142135623730951

CODE:

```
#include <stdio.h>
#include <math.h>

/* Euclidean distance between two points */
double dist(double x1, double y1, double x2, double y2) {
   return sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2));
}

int main() {
   int n, i, j, a = 0, b = 1;
   double x[50], y[50], d, min;
   printf("Enter number of points: ");
   scanf("%d", &n);
   for (i = 0; i < n; i++) {
      printf("Enter point %d (x y): ", i + 1);
      scanf("%lf %lf", &x[i], &y[i]);
   }

   min = dist(x[0], y[0], x[1], y[1]);
```

```
    for (i = 0; i < n; i++)
        for (j = i + 1; j < n; j++) {          /* every pair */
            d = dist(x[i], y[i], x[j], y[j]);
            if (d < min) {
                min = d;
                a = i;
                b = j;
            }
        }

    printf("Closest pair: (%g, %g) - ", x[a], y[a]);
    printf("(%g, %g)\n", x[b], y[b]);
    printf("Minimum distance: %f\n", min);
    return 0;
}
```

OUTPUT:

```
Enter number of points: 4
Enter point 1 (x y): 1 2
Enter point 2 (x y): 4 5
Enter point 3 (x y): 7 8
Enter point 4 (x y): 3 1
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.236068
```

Time Complexity: $O(n^2)$ | Space Complexity: O(1)

26. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the sameline?

Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5).

output: P3, P4, P6, P5, P7, P1

CODE:

```
#include <stdio.h>
#include <math.h>

#define EPS 1e-9
int n;
double x[20], y[20];

/* Euclidean distance between point a and point b */
```

```
double dist(int a, int b) {
   double dx = x[a] - x[b], dy = y[a] - y[b];
   return sqrt(dx * dx + dy * dy);
}

/* > 0 left of i->j, < 0 right of i->j, 0 same line */
double cross(int i, int j, int k) {
   return (x[j] - x[i]) * (y[k] - y[i])
        - (y[j] - y[i]) * (x[k] - x[i]);
}

/* 1 if collinear point k lies between i and j */
int between(int i, int j, int k) {
   double dx = x[j] - x[i], dy = y[j] - y[i];
   double dot = (x[k] - x[i]) * dx + (y[k] - y[i]) * dy;
   return dot >= -EPS && dot <= dx * dx + dy * dy + EPS;
}

/* brute force closest pair: O(n^2) */
void closestPair() {
   int i, j, a = 0, b = 1;
   double d, min = dist(0, 1);
   for (i = 0; i < n; i++)
      for (j = i + 1; j < n; j++) {
         d = dist(i, j);
         if (d < min) {
            min = d;
            a = i;
            b = j;
         }
      }
   printf("Closest pair: P%d (%g, %g) - ", a + 1, x[a], y[a]);
   printf("P%d (%g, %g)\n", b + 1, x[b], y[b]);
   printf("Minimum distance: %f\n", min);
}

/* brute force convex hull: O(n^3) */
void convexHull() {
   int next[20], i, j, k, valid, start = 0, p;
   double c;
   for (i = 0; i < n; i++) next[i] = -1;

   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++) {
         if (i == j) continue;
         valid = 1;
         for (k = 0; k < n && valid; k++) {
            if (k == i || k == j) continue;
            c = cross(i, j, k);
            if (c < -EPS)
               valid = 0;            /* k on the right */
            else if (c <= EPS && !between(i, j, k))
               valid = 0;            /* same line, outside */
         }
         if (valid) next[i] = j;       /* i -> j is a hull edge */
      }

   for (i = 1; i < n; i++)
```

```
        if (x[i] < x[start] || (x[i] == x[start] && y[i] < y[start]))
            start = i;

    printf("Convex Hull (counter-clockwise): ");
    p = start;
    do {
        printf("P%d", p + 1);
        p = next[p];
        if (p != -1 && p != start) printf(", ");
    } while (p != -1 && p != start);
    printf("\n");
}

int main() {
    int i;
    printf("Enter number of points: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter point %d (x y): ", i + 1);
        scanf("%lf %lf", &x[i], &y[i]);
    }
    closestPair();
    convexHull();
    return 0;
}
```

OUTPUT:

```
Enter number of points: 8
Enter point 1 (x y): 10 0
Enter point 2 (x y): 11 5
Enter point 3 (x y): 5 3
Enter point 4 (x y): 9 3.5
Enter point 5 (x y): 15 3
Enter point 6 (x y): 12.5 7
Enter point 7 (x y): 6 6.5
Enter point 8 (x y): 7.5 4.5
Closest pair: P4 (9, 3.5) - P8 (7.5, 4.5)
Minimum distance: 1.802776
Convex Hull (counter-clockwise): P3, P1, P5, P6, P7
```

27. Write a program that finds the convex hull of a set of 2D points using the brute force approach.

Input:

A list or array of points represented by coordinates (x, y).

Points: [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]

Output:

The list of points that form the convex hull in counter-clockwise order.

Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]

CODE:

```
#include <stdio.h>

#define EPS 1e-9
int n;
double x[20], y[20];

/* > 0 left of i->j, < 0 right of i->j, 0 same line */
double cross(int i, int j, int k) {
   return (x[j] - x[i]) * (y[k] - y[i])
       - (y[j] - y[i]) * (x[k] - x[i]);
}

/* 1 if collinear point k lies between i and j */
int between(int i, int j, int k) {
   double dx = x[j] - x[i], dy = y[j] - y[i];
   double dot = (x[k] - x[i]) * dx + (y[k] - y[i]) * dy;
   return dot >= -EPS && dot <= dx * dx + dy * dy + EPS;
}

int main() {
   int i, j, k, valid, start = 0, p, next[20];
   double c;
   printf("Enter number of points: ");
   scanf("%d", &n);
   for (i = 0; i < n; i++) {
      printf("Enter point %d (x y): ", i + 1);
      scanf("%lf %lf", &x[i], &y[i]);
      next[i] = -1;
   }

   /* i -> j is a hull edge if no point lies on its right side */
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++) {
         if (i == j) continue;
         valid = 1;
         for (k = 0; k < n && valid; k++) {
            if (k == i || k == j) continue;
            c = cross(i, j, k);
            if (c < -EPS)
               valid = 0;
            else if (c <= EPS && !between(i, j, k))
               valid = 0;
         }
         if (valid) next[i] = j;
      }
```

```
    /* start from the left-most point and follow the edges */
    for (i = 1; i < n; i++)
        if (x[i] < x[start] || (x[i] == x[start] && y[i] < y[start]))
            start = i;

    printf("Convex Hull: [");
    p = start;
    do {
        printf("(%g, %g)", x[p], y[p]);
        p = next[p];
        if (p != -1 && p != start) printf(", ");
    } while (p != -1 && p != start);
    printf("]\n");
    return 0;
}
```

OUTPUT:

```
Enter number of points: 5
Enter point 1 (x y): 1 1
Enter point 2 (x y): 4 6
Enter point 3 (x y): 8 1
Enter point 4 (x y): 0 0
Enter point 5 (x y): 3 3
Convex Hull: [(0, 0), (8, 1), (4, 6)]
```

Time Complexity: $O(n^3)$ | Space Complexity: O(n)

28. You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP. The program should:

1. Define a function distance(city1, city2) to calculate the distance between two cities (e.g., Euclidean distance).

2. Implement a function tsp(cities) that takes a list of cities as input and performs the following:

- Generate all possible permutations of the cities (excluding the starting city) using itertools.permutations.
- For each permutation (representing a potential route):
- Calculate the total distance traveled by iterating through the path and summing the distances between consecutive cities.
- Keep track of the shortest distance encountered and the corresponding path.
- Return the minimum distance and the shortest path (including the starting city at the beginning and end).

3. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

Simple Case: Four cities with basic coordinates (e.g., [(1, 2), (4, 5), (7, 1), (3, 6)])

More Complex Case: Five cities with more intricate coordinates (e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

Test Case 1:

Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]

CODE:

```
#include <stdio.h>
#include <math.h>

int n, perm[10], used[10], best[10];
double x[10], y[10], bestDist = 1e18;

/* Euclidean distance between city a and city b */
double distance(int a, int b) {
   double dx = x[a] - x[b], dy = y[a] - y[b];
   return sqrt(dx * dx + dy * dy);
}

/* total length of the route in perm[], returning to the start city */
double routeLength() {
   int i;
   double total = 0;
   for (i = 0; i < n - 1; i++)
      total += distance(perm[i], perm[i + 1]);
   total += distance(perm[n - 1], perm[0]);
   return total;
}

/* all permutations of the cities (city 0 is the start) */
void permute(int pos) {
   int i;
   double d;
   if (pos == n) {
      d = routeLength();
      if (d < bestDist) {
         bestDist = d;
         for (i = 0; i < n; i++) best[i] = perm[i];
      }
      return;
   }
   for (i = 1; i < n; i++)
      if (!used[i]) {
         used[i] = 1;
         perm[pos] = i;
         permute(pos + 1);
         used[i] = 0;
      }
}
```

```
int main() {
    int i;
    printf("Enter number of cities: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter city %d (x y): ", i + 1);
        scanf("%lf %lf", &x[i], &y[i]);
    }
    perm[0] = 0;
    used[0] = 1;
    permute(1);

    printf("Shortest Distance: %f\n", bestDist);
    printf("Shortest Path: [");
    for (i = 0; i < n; i++)
        printf("(%g, %g), ", x[best[i]], y[best[i]]);
    printf("(%g, %g)]\n", x[best[0]], y[best[0]]);
    return 0;
}
```

OUTPUT:

```
Enter number of cities: 4
Enter city 1 (x y): 1 2
Enter city 2 (x y): 4 5
Enter city 3 (x y): 7 1
Enter city 4 (x y): 3 6
Shortest Distance: 16.969112
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]
```

```
Enter number of cities: 5
Enter city 1 (x y): 2 4
Enter city 2 (x y): 8 1
Enter city 3 (x y): 1 7
Enter city 4 (x y): 6 3
Enter city 5 (x y): 5 9
Shortest Distance: 23.129950
Shortest Path: [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)]
```

Time Complexity: O((n-1)! × n) | Space Complexity: O(n)

29. You are given a cost matrix where each element cost[i][j] represents the cost of assigning worker i to task j. Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function total_cost(assignment, cost_matrix) that takes

an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function assignment_problem(cost_matrix) that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).

Test Cases:
Input
Simple Case: Cost Matrix:
[[3, 10, 7],
[8, 5, 12],
[4, 6, 9]]
More Complex Case: Cost Matrix:
[[15, 9, 4],
[8, 7, 18],
[6, 12, 11]]
Output:
Test Case 1:
Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)]
Total Cost: 19
Test Case 2:
Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]
Total Cost: 24

CODE:

```
#include <stdio.h>

int n, cost[10][10], perm[10], used[10], best[10], bestCost = 1000000;

/* assignment[i] = task given to worker i */
int totalCost(int assignment[], int c[][10], int size) {
   int i, total = 0;
   for (i = 0; i < size; i++) total += c[i][assignment[i]];
   return total;
}

/* try every permutation of tasks */
void permute(int pos) {
   int i, t;
   if (pos == n) {
      t = totalCost(perm, cost, n);
      if (t < bestCost) {
         bestCost = t;
         for (i = 0; i < n; i++) best[i] = perm[i];
      }
      return;
   }
   for (i = 0; i < n; i++)
      if (!used[i]) {
         used[i] = 1;
```

```
            perm[pos] = i;
            permute(pos + 1);
            used[i] = 0;
        }
}

int main() {
    int i, j;
    printf("Enter n: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter row %d of cost matrix: ", i + 1);
        for (j = 0; j < n; j++) scanf("%d", &cost[i][j]);
    }
    permute(0);

    printf("Optimal Assignment: [");
    for (i = 0; i < n; i++) {
        printf("(worker %d, task %d)", i + 1, best[i] + 1);
        if (i < n - 1) printf(", ");
    }
    printf("]\n");
    printf("Total Cost: %d\n", bestCost);
    return 0;
}
```

OUTPUT:

```
Enter n: 3
Enter row 1 of cost matrix: 3 10 7
Enter row 2 of cost matrix: 8 5 12
Enter row 3 of cost matrix: 4 6 9
Optimal Assignment: [(worker 1, task 3), (worker 2, task 2), (worker 3, task 1)]
Total Cost: 16
```

```
Enter n: 3
Enter row 1 of cost matrix: 15 9 4
Enter row 2 of cost matrix: 8 7 18
Enter row 3 of cost matrix: 6 12 11
Optimal Assignment: [(worker 1, task 3), (worker 2, task 2), (worker 3, task 1)]
Total Cost: 17
```

Time Complexity: O(n! × n) | Space Complexity: O(n)

30. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

Define a function total_value(items, values) that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.

Define a function is_feasible(items, weights, capacity) that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

Test Cases:

Simple Case:

Items: 3 (represented by indices 0, 1, 2)

Weights: [2, 3, 1]

Values: [4, 5, 3]

Capacity: 4

More Complex Case:

Items: 4 (represented by indices 0, 1, 2, 3)

Weights: [1, 2, 3, 4]

Values: [2, 4, 6, 3]

Capacity: 6

Output:

Test Case 1:

Optimal Selection: [0, 2] (Items with indices 0 and 2)

Total Value: 7

Test Case 2:

Optimal Selection: [0, 1, 2] (Items with indices 0, 1, and 2)

Total Value: 10

CODE:

```
#include <stdio.h>

/* sum of the values of the selected items */
int totalValue(int items[], int count, int values[]) {
   int i, total = 0;
   for (i = 0; i < count; i++) total += values[items[i]];
   return total;
}

/* 1 if the total weight of the selected items fits in the knapsack */
int isFeasible(int items[], int count, int weights[], int capacity) {
   int i, w = 0;
   for (i = 0; i < count; i++) w += weights[items[i]];
   return w <= capacity;
}

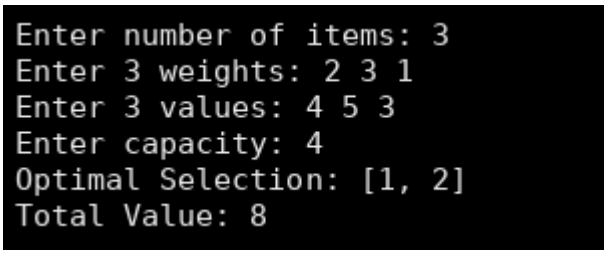
int main() {
   int n, i, mask, count, cap, val, bestVal = -1, bestCount = 0;
   int w[20], v[20], sel[20], best[20];
   printf("Enter number of items: ");
   scanf("%d", &n);
   printf("Enter %d weights: ", n);
```

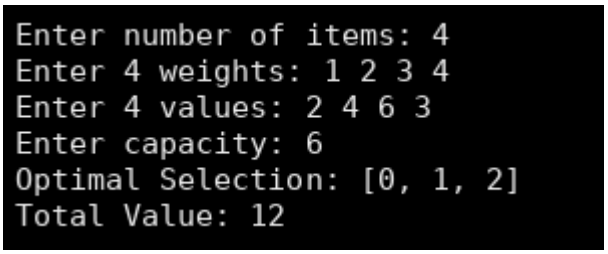
```
    for (i = 0; i < n; i++) scanf("%d", &w[i]);
    printf("Enter %d values: ", n);
    for (i = 0; i < n; i++) scanf("%d", &v[i]);
    printf("Enter capacity: ");
    scanf("%d", &cap);

    for (mask = 0; mask < (1 << n); mask++) {   /* all 2^n subsets */
        count = 0;
        for (i = 0; i < n; i++)
            if (mask & (1 << i)) sel[count++] = i;
        if (isFeasible(sel, count, w, cap)) {
            val = totalValue(sel, count, v);
            if (val > bestVal) {
                bestVal = val;
                bestCount = count;
                for (i = 0; i < count; i++) best[i] = sel[i];
            }
        }
    }

    printf("Optimal Selection: [");
    for (i = 0; i < bestCount; i++) {
        printf("%d", best[i]);
        if (i < bestCount - 1) printf(", ");
    }
    printf("]\n");
    printf("Total Value: %d\n", bestVal);
    return 0;
}
```

OUTPUT:





Time Complexity: $O(2^n \times n)$ | Space Complexity: O(n)